Короткий вывод

$$B = \texttt{Stream2+Stream3}: \quad \text{expand/hash/dedup/NCCL/prune/rebuild/compact}$$

Главный bottleneck не `CUTLASS GEMM`, а нерегулярный candidate path:

$$N_{\text{cand}} = N_{\text{active}} \cdot F, \qquad F = 24$$

$$T_{\text{depth}} \approx \max(T_{\text{infer}}, T_{\text{cand}} + T_{\text{net}}) + T_{\text{prune}} + T_{\text{rebuild}} + T_{\text{compact}}$$

$$T_{\text{cand}} \sim N_{\text{cand}} \cdot (120\text{B apply} + 120\text{B hash} + \alpha_{\text{atomic}} P), \qquad T_{\text{net}} \sim N_{\text{tiles}} \cdot W \cdot C_{\text{bucket}}$$

Пруфы по коду

Stream1: `beam_engine.cpp:383--408`

Статический scorer уже делает embedding + CUTLASS GEMM + quantize:

$$\texttt{embed} \to \texttt{GEMM}_{1536\times512} \to 4 \cdot \texttt{GEMM}_{512\times512} \to \texttt{GEMM}_{512\times24}$$

Stream2: `beam_engine.cpp:1107--1138`

После каждого microbatch Stream2 обязан обработать

$$L_{\text{tile}} \le K_{\text{expand\_tile}}, \qquad L_{\text{total}} = \texttt{micro\_size} \cdot 24$$

через `launch_process_score_slot`: state materialization, hash table, histogram, remote bucket packing.

Hash hot path: `beam_kernels.cu:351--488`

Hash insert/update использует probing и atomics:

$$P \le \texttt{PROBE\_LIMIT}, \qquad \text{cost} \propto N_{\text{cand}} \cdot E[P] + \texttt{atomicAdd/CAS}$$

Stream3 all-to-all: `beam_engine.cpp:972--986`

Каждый tile делает grouped NCCL counts + payload exchange:

$$V_{\text{tile}} = \texttt{world\_size} \cdot \texttt{bucket\_cap\_per\_peer} \cdot sizeof(\texttt{CandidateRecord})$$

Threshold/prune/rebuild: `beam_engine.cpp:989--1032`

Каждый threshold update делает histogram allreduce, prune, clear hash, rebuild:

$$T_{\text{threshold}} \approx T_{\text{allreduce}}(65536) + T_{\text{prune}}(K_{\text{work}}) + T_{\text{clear}}(H) + T_{\text{rebuild}}(K_{\text{work}})$$

status sync: `beam_engine.cpp:766--790`

Логирование ждёт все streams:

$$\texttt{status()} = \texttt{synchronize(Stream1, lanes, Stream2, Stream3)}$$

Пруфы по истории измерений

`docs/PROJECT_MEMORY.md:5--11`

Добавлен `DEPTH_TUNING`: $wall\_ms$, num_micro, $expand\_tiles$, counters. Прямой смысл: искать задержку через tile/counter нагрузку Stream2/3.

`docs/PROJECT_MEMORY.md:37`

Зафиксировано:

$$\text{dominant\_cost} = \text{hash\_clear} + K_{\text{work}}\text{-scan} + N_{\text{local}}\text{-check} + \text{NCCL} + \text{histogram} \neq \text{neural GEMM}$$

`docs/CUTLASS_STREAM2_STREAM3_REVIEW.md:7--15`

CUTLASS применим к Stream1; Stream2/3 = irregular memory/hash/NCCL, не GEMM.

Итог

$$\boxed{\text{bottleneck} = \texttt{Stream2/Stream3 candidate pipeline}}$$

$$\boxed{\text{первый рычаг} = \text{уменьшить } N_{\text{cand}}, E[P], V_{\text{tile}}, K_{\text{work}}\text{-passes}}$$